

수달의 한마디: React Hook 탐구

작은 명언 앱 안에서 Hook 나누어 쓰기

전체 Hook 사용 지도

OtterAdviceApp.jsx

useState
useEffect
useCallback

useSlideshow.js

useReducer
useRef
useMemo
useEffect
useCallback

OtterGallery.jsx

useId
useSlideshow

1. 기초 상태와 외부 통신

useState

단순 화면 상태

useEffect

초기 API 호출

```
// UI state variables
const [quote, setQuote] = useState(defaultQuote);

// Fetch API on initial load
useEffect(() => {
  const controller = new AbortController();
  loadAdvice(controller.signal);

  return () => controller.abort();
}, [loadAdvice]);
```

2. useReducer (1/2) : 로직 캡슐화

상태 전환 규칙 정의

index

playing

```
// Handle complex state transitions in one place
function reducer(state, action) {
  switch (action.type) {
    case 'next':
      return { ...state, index: (state.index + 1) % action.count };
    case 'prev':
      return {
        ...state,
        index: (state.index - 1 + action.count) % action.count
      };
    case 'toggle':
      return { ...state, playing: !state.playing };
  }
}
```

2. useReducer (2/2) : 액션 전달

단일 상태 제어

dispatch

UI 단순화

```
// Connect state and reducer
```

```
const [state, dispatch] = useReducer(reducer, { index: 0, playing: true });
```

```
// Dispatch actions to update state
```

```
const next = () ⇒ dispatch({ type: 'next', count });
```

```
const toggle = () ⇒ dispatch({ type: 'toggle' });
```

```
// Call next() from UI
```

```
<button onClick={next}>다음 사진</button>
```

3. useRef + useMemo

렌더링 무관 값

타이머 ID

계산값 보존

현재 사진 데이터

```
// Store timer ID without re-rendering
```

```
const timerRef = useRef(null);
```

```
// Recalculate only when dependencies change
```

```
const current = useMemo(  
  () => (count > 0 ? items[state.index % count] : null),  
  [items, state.index, count]  
);
```

4. useId + Custom Hook

고유 ID

접근성 연결

구조 분리

UI와 비즈니스 로직

```
// Generate stable ID for accessibility
```

```
const titleId = useId();
```

```
// Extract slideshow logic into a custom hook
```

```
const {
```

```
  index, playing, current, count,
```

```
  next, prev, go, toggle
```

```
} = useSlideshow(photos);
```

발표 결론

useState

화면에 즉각적으로 반영되는 단순 상태 관리

useEffect

컴포넌트 렌더링 이후 외부 작업(API) 처리

useCallback

함수 재생성을 방지하여 렌더링과 의존성 안정화

useReducer

여러 액션이 얹힌 복잡한 상태 전환 캡슐화

useRef

리렌더링을 발생시키지 않는 내부 값(타이머) 보존

useMemo

상태에서 파생되는 데이터를 캐싱하여 재계산 방지

useId

웹 접근성 확보를 위한 안정적인 고유 ID 생성

Custom Hook

UI와 비즈니스 로직을 분리하여 모듈화